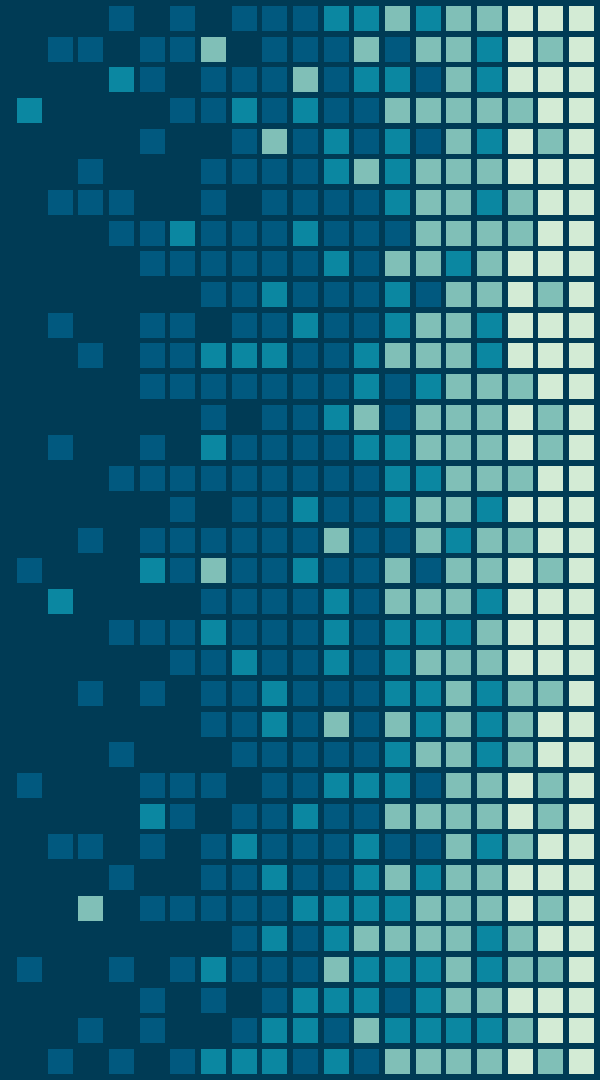
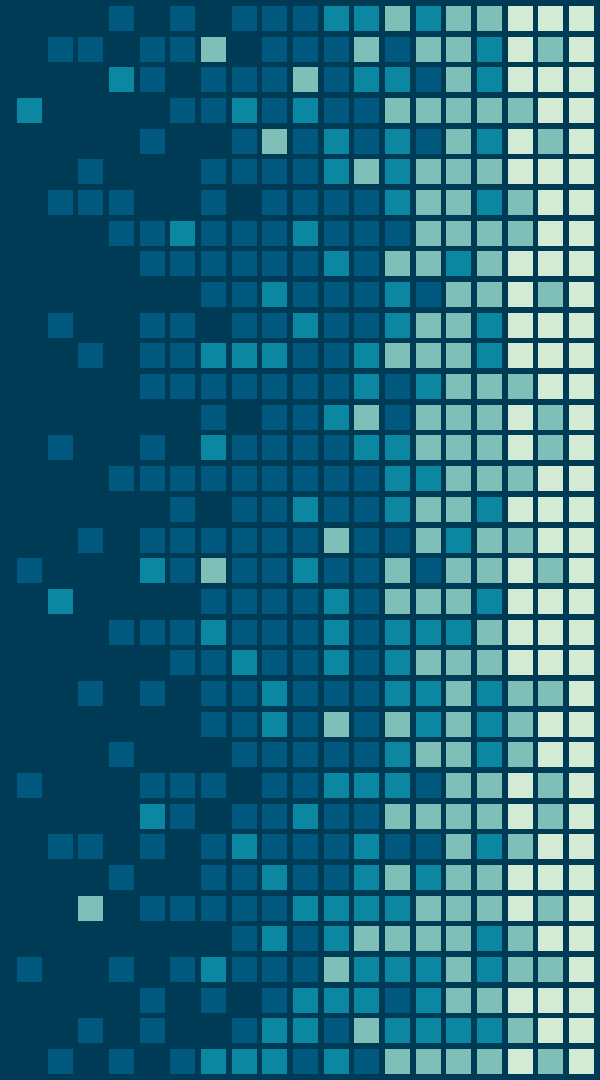


ASL Hand Sign Detector Tensor Titans

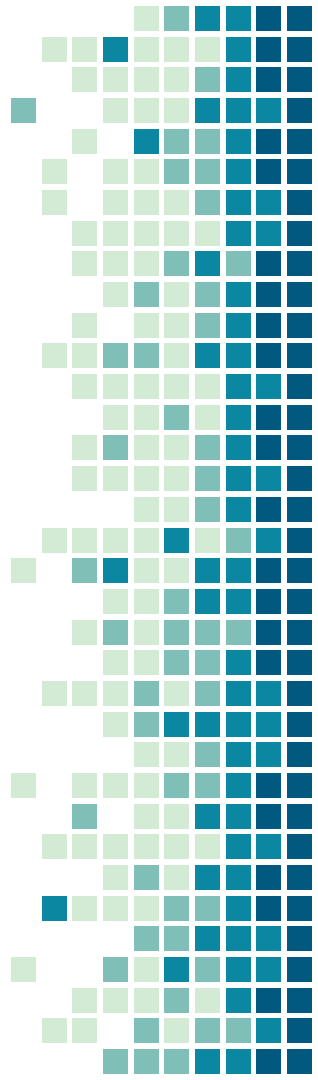


Project Overview



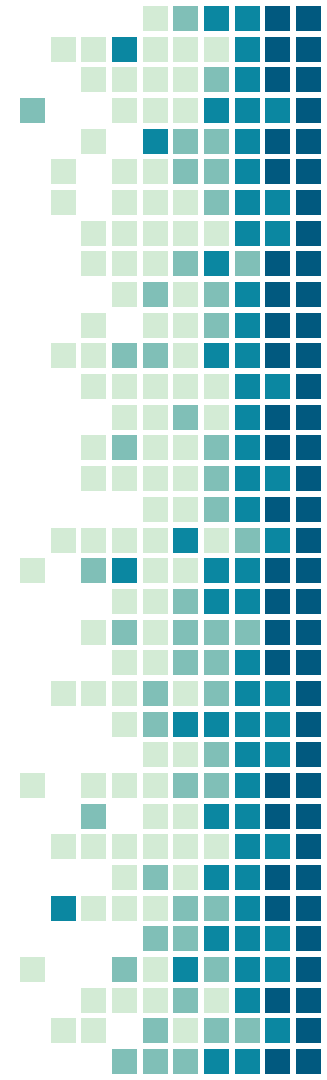
Project Goal

- Use Machine Learning to create a model that can recognize American Sign Language (ASL) letters in real time



Project Roadmap

- Find a Dataset and Organize
 - <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>
 - Used Letters A-E
 - 500 Images for Each Letter, 2500 Images Total
- Data Preprocessing
 - Split Into Training and Testing Sets
- Training Model
 - Premade Model Using Tensorflow
- Use Webcam to Detect Hand Gestures
 - Real-Time Prediction Using Mediapipe and OpenCV

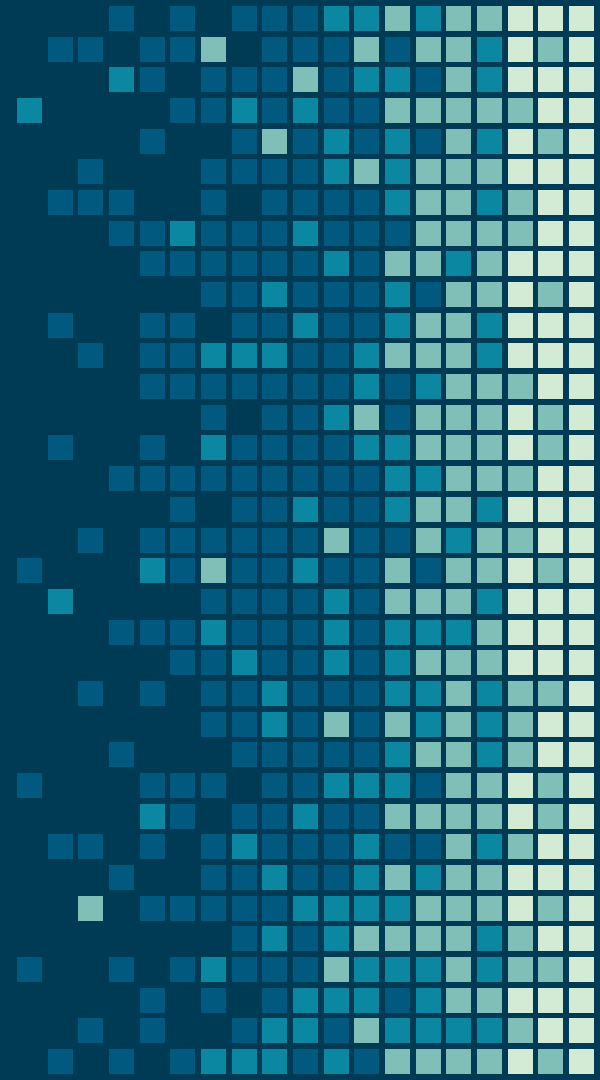


Tools Used

- Python
- Tensorflow
- Google Colab
- VS Code
- Cv2
- Mediapipe
- Numpy
- Github



Data Processing Pipeline



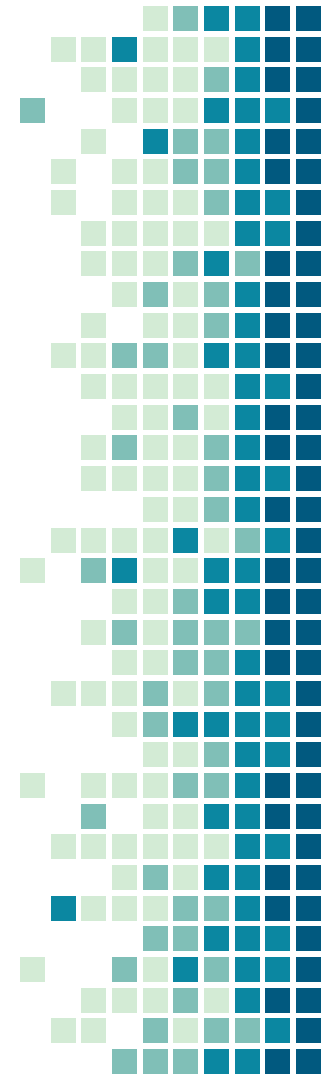
What is Data Processing Pipeline?

A data processing pipeline is a series of steps that transform raw data into clean later structures the data that a ML model can use to learn and make predictions.

Dataset Preparation

Typical Stages in a ML Data Pipeline:

1. **Data Collection** – Get data from files, sensors, APIs, etc.
2. **Data Cleaning** – Handle missing values, noise, or duplicates
3. **Preprocessing** – Resize, normalize, encode, etc.
4. **Splitting** – Divide into train/test sets
5. **Augmentation (optional)** – Generate more data artificially
6. **Feeding to Model** – Load batches for training or prediction



Data Collection:

- We collected pre-labeled ASL images (A–E) from **Kaggle**
- **2500** total images (**500 per class**)

Data Cleaning:

- The dataset was already clean and labeled
- No need to remove corrupt or mislabeled images

Preprocessing:

- Resized all images to **224×224** (required by MobileNetV2)
- Normalized pixel values

Splitting:

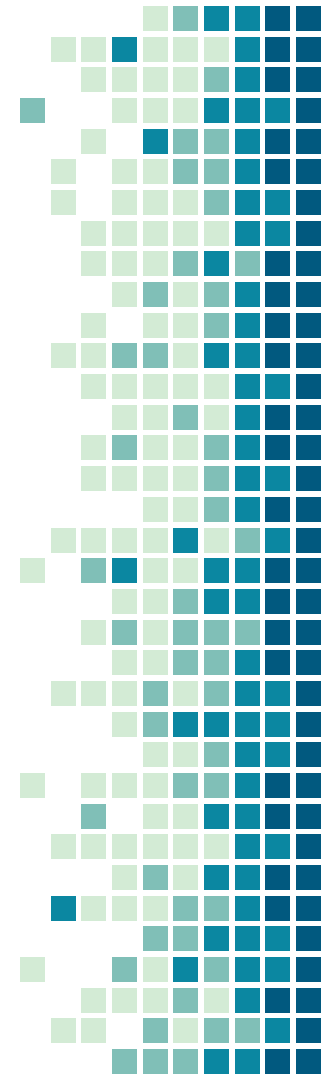
- **80%** of images used for training
- **20%** used for testing
- Folder structure handled the split (**train/** and **test/**)

Augmentation:

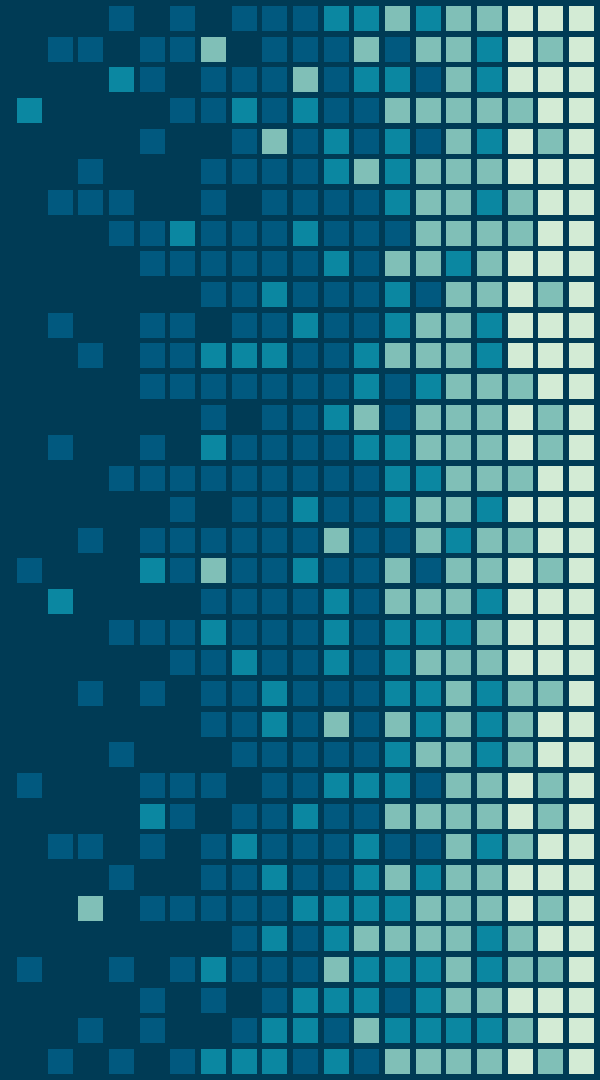
- We Create new variations to boost training
- Applied rotation, zoom, and shift to training data helped to simulate different hand positions

Feeding to Model

- Passed data into MobileNetV2 for training



MODEL TRAINING



Model

- Pre-trained model - MobileNetV2 (image classification & object detection)
- Froze base to keep pre-learned image features
- Added custom layers
 - GlobalAveragePooling2D() - lowers dimensions to single value
 - Dense(128, activation='relu') - detects features unique to ASL letters
 - Dropout(0.3) & Dropout(0.2) - prevents overfitting
 - Dense(5, activation = 'softmax') - classifies input into one of five ASL letters: A, B, C, D, or E

MobileNetV2 → Pooling → Dense → Dropout → Softmax

Model Training

#Compile the model

- Loss Function: 'sparse_categorical_crossentropy'
 - Measures guess to actual label
 - Multiple classes as integers (0-4, for A - E)
- Optimizer: 'adam'
 - Helps model learn + improve during training
 - Adjusts learning rates automatically
- Metric: 'accuracy'
 - Evaluates how often model predicts correct letter
- Epochs: 15
 - Cycles which go through the entire training dataset
 - Train for 15 epochs to let model improve

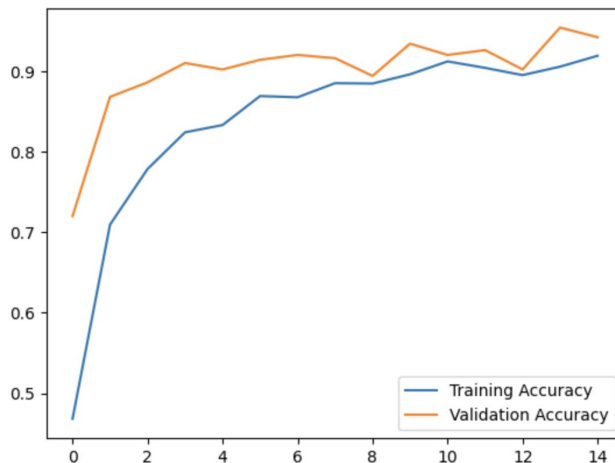
```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
history = model.fit(  
    train_data,  
    epochs=15,  
    validation_data=test_data  
)
```

Training Results

- Training accuracy = how well the model has learned the patterns in the training dataset
- Validation accuracy = how well the model generalizes to new data

Epoch 1/15	113s	2s/step	- accuracy: 0.3631 - loss: 1.6038 - val_accuracy: 0.7200 - val_loss: 0.7510
Epoch 2/15	153s	2s/step	- accuracy: 0.6707 - loss: 0.8317 - val_accuracy: 0.8680 - val_loss: 0.4455
Epoch 3/15	119s	2s/step	- accuracy: 0.7726 - loss: 0.5987 - val_accuracy: 0.8860 - val_loss: 0.3759
Epoch 4/15	146s	2s/step	- accuracy: 0.8029 - loss: 0.5098 - val_accuracy: 0.9100 - val_loss: 0.3157
Epoch 5/15	147s	2s/step	- accuracy: 0.8343 - loss: 0.4549 - val_accuracy: 0.9020 - val_loss: 0.3045
Epoch 6/15	139s	2s/step	- accuracy: 0.8576 - loss: 0.4007 - val_accuracy: 0.9140 - val_loss: 0.2515
Epoch 7/15	139s	2s/step	- accuracy: 0.8635 - loss: 0.3661 - val_accuracy: 0.9200 - val_loss: 0.2298
Epoch 8/15	141s	2s/step	- accuracy: 0.8818 - loss: 0.3331 - val_accuracy: 0.9160 - val_loss: 0.2637
Epoch 9/15	161s	2s/step	- accuracy: 0.8840 - loss: 0.3331 - val_accuracy: 0.8940 - val_loss: 0.2562
Epoch 10/15	124s	2s/step	- accuracy: 0.8912 - loss: 0.3006 - val_accuracy: 0.9340 - val_loss: 0.1861
Epoch 11/15	143s	2s/step	- accuracy: 0.9100 - loss: 0.2618 - val_accuracy: 0.9200 - val_loss: 0.2027
Epoch 12/15	141s	2s/step	- accuracy: 0.8931 - loss: 0.2956 - val_accuracy: 0.9260 - val_loss: 0.1914
Epoch 13/15	142s	2s/step	- accuracy: 0.8949 - loss: 0.2862 - val_accuracy: 0.9020 - val_loss: 0.2603
Epoch 14/15	141s	2s/step	- accuracy: 0.8945 - loss: 0.2907 - val_accuracy: 0.9540 - val_loss: 0.1534
Epoch 15/15	142s	2s/step	- accuracy: 0.9137 - loss: 0.2343 - val_accuracy: 0.9420 - val_loss: 0.1571



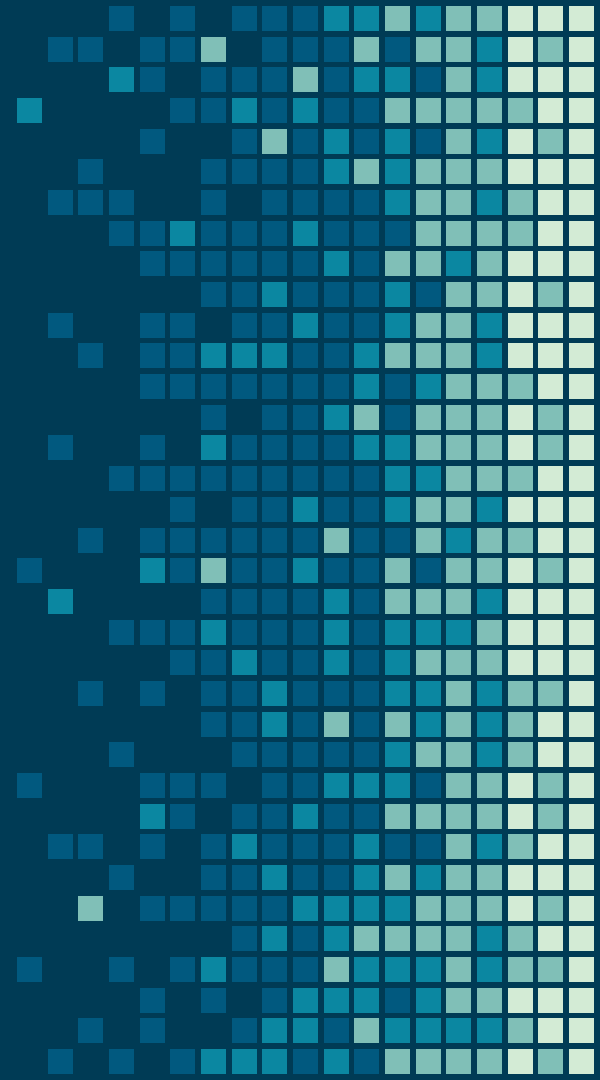
- Training and validation accuracy improved steadily across 15 epochs
 - Final: Training accuracy of 91% and validation accuracy of 94%

Real-time Prediction Using Webcam

- Saved model and loaded in VS Code
- Used MediaPipe to detect hand landmarks from webcam (OpenCV)
- Cropped the hand region, resized (224 x 224), and normalized
 - Adjusted pixel values to certain range (prevents from dominating the training data)
- Model predicts ASL letter (A-E) & overlays result on video

```
hand_roi = cv2.resize(hand_roi, (224, 224))  
hand_roi = np.expand_dims(hand_roi, axis=0) / 255.0  
prediction = model.predict(hand_roi)  
label = labels[np.argmax(prediction)]
```

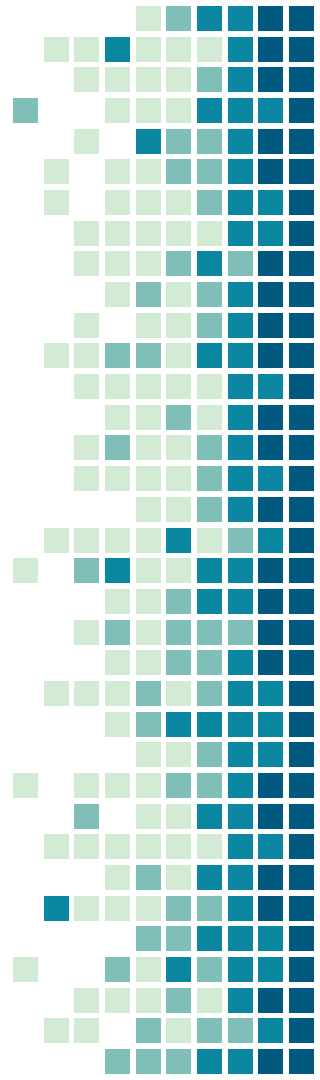
Challenges faced & learnings



Challenges Faced

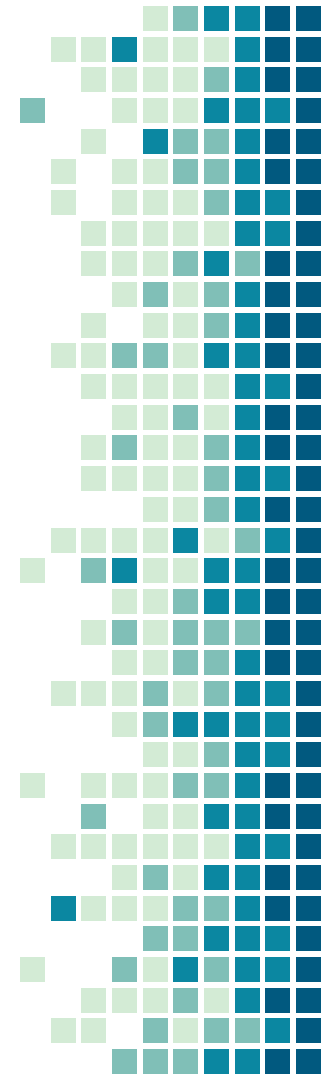
1. **Deciding the Data Size and format:** Our data included pictures of hand signs so we had to decide on whether clicking our own pictures or use available data sets then formatting them as needed.

Since We were using the free version of Colab so we decided to go for the first 5 letters as it would not be able to handle more photosets and labels
2. **Dependency Conflicts:** Faced version mismatches between TensorFlow, Keras, NumPy, and MediaPipe, requiring careful installation and downgrades to maintain compatibility.
3. **Switching platforms** midway from Colab to VS Code took some time but it was necessary as after we trained the model we had to check its performance with live camera detection using openCV which was not supported in Colab.
4. **Retraining the model:** b and d were not detecting during the live camera test and the model was getting confused between A and E.

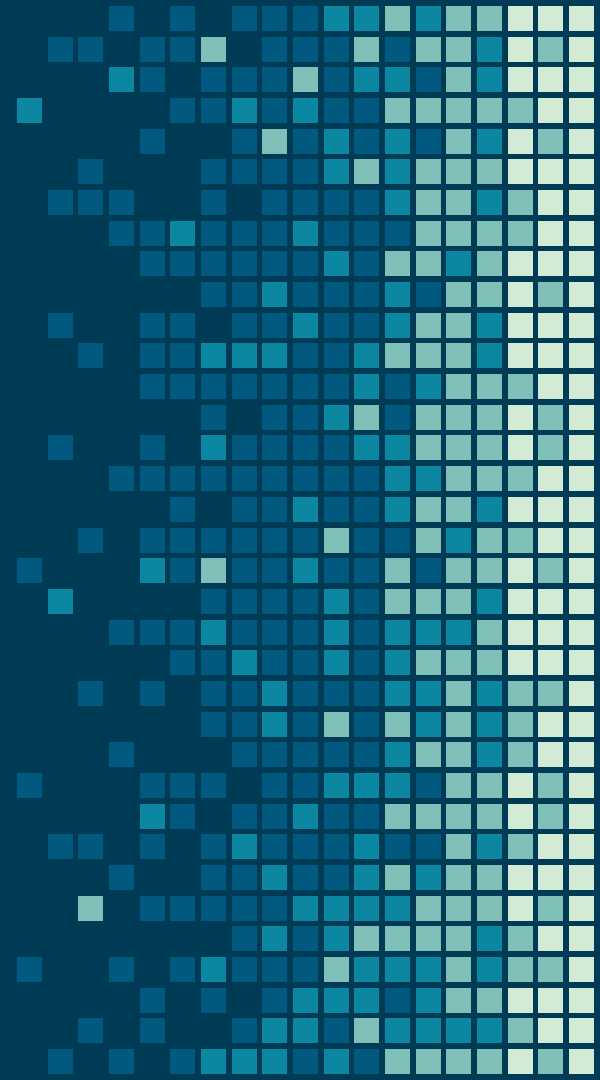


Learnings

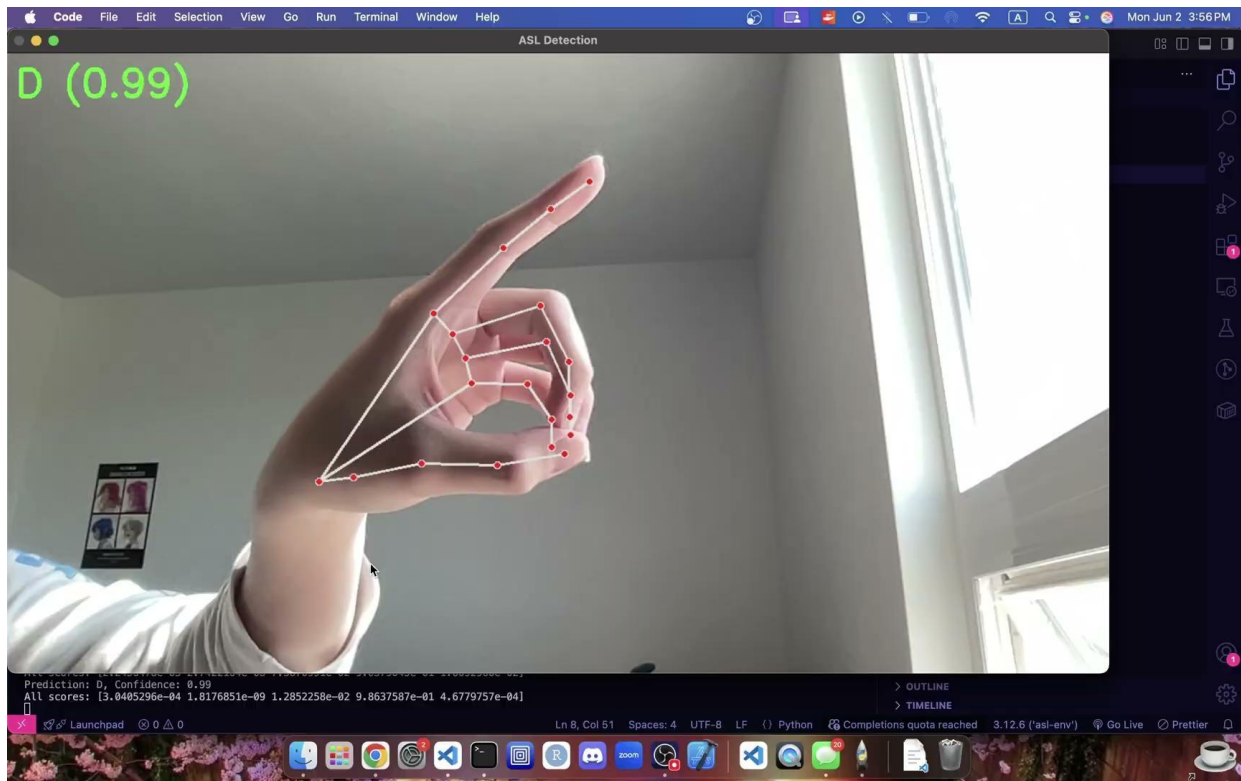
- Learned to preprocess and organize image datasets for classification tasks.
- Applied data augmentation (rotation, zoom, flip) to improve model accuracy.
- Gained hands-on experience with TensorFlow, Keras, OpenCV, and MediaPipe.
- Understood the impact of class imbalance and how to handle it.
- Integrated real-time webcam input with live predictions.
- Solved library compatibility and environment setup issues.
- Improved debugging, teamwork, and iteration skills throughout the project.



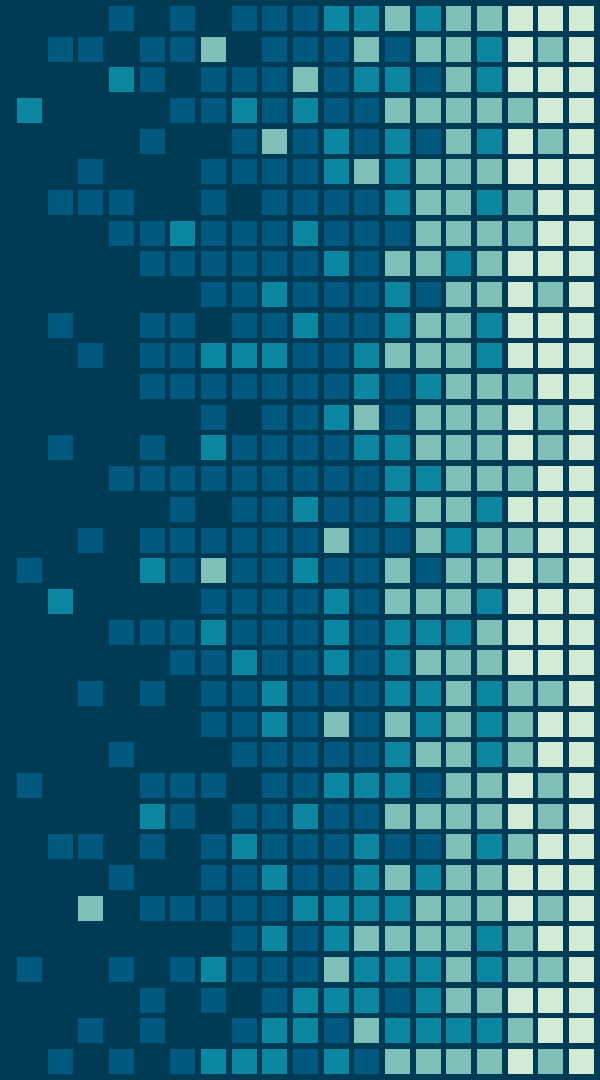
Demo of Model



Demo Video

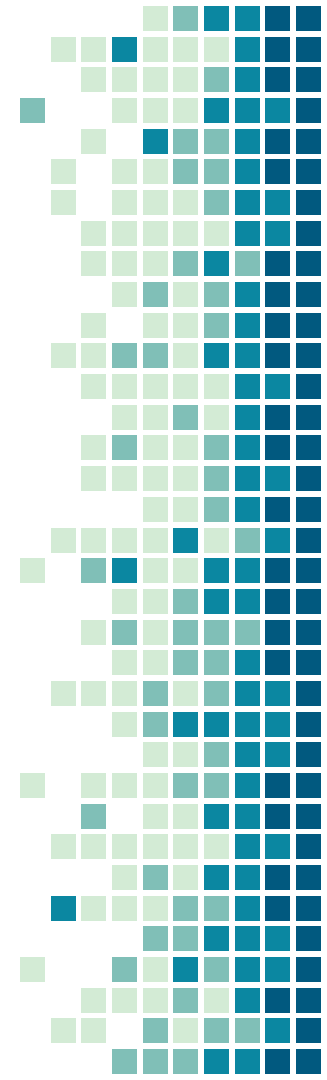


Future Work



Future Work

- Expand to All 26 Letters
 - Include Simple ASL Words or Sentences
 - Expand Dataset to Include More Images
- Improve Model Accuracy
 - Include a Feedback Loop
 - Other Fine Tuning Adjustments



Thank You!

